

Dataflow Analysis Complexity and Correctness

The termination of the worklist algorithm for dataflow analysis relies on two conditions: the dataflow lattice having finite height and the flow functions being monotonic. A flow function f is monotonic iff $\sigma_1 \sqsubseteq \sigma_2$ implies $f(\sigma_1) \sqsubseteq f(\sigma_2)$ for all σ_1, σ_2 .

The correctness of a dataflow analysis depends on the local soundness of its flow functions. For every program configuration c_i in the trace of a program P , a flow function f is locally sound iff $(P \vdash c_i \rightsquigarrow c_{i+1})$ and $\alpha(c_i) \sqsubseteq \sigma_{n_i}$ and $f[P[n_i]](\sigma_{n_i}) = \sigma_{n_{i+1}} \Rightarrow \alpha(c_{i+1}) \sqsubseteq \sigma_{n_{i+1}}$.

Exercises

These exercises prove properties of parity analysis. Assume the following:

- A lattice $(L, \sqsubseteq)$ where $L = \{\top, O, V, \perp\}$ and $\perp \sqsubseteq \{O, V\} \sqsubseteq \top, O \sqcup V = \top$
- An abstraction function $\alpha : \mathbb{Z} \mapsto L$, defined as follows:

$$\alpha(n) = \begin{cases} V & \text{when } n \text{ is an even integer } (n \in \{2k : k \in \mathbb{Z}\}) \\ O & \text{when } n \text{ is an odd integer } (n \in \{2k + 1 : k \in \mathbb{Z}\}) \end{cases}$$

- a flow function f_P
- initial dataflow analysis assumptions σ_0 , in this case σ_0 maps all variables' initial states to $\top$.

1. Disprove the local soundness of the incorrect flow function $f_P[a := b](\sigma) = \sigma[a \mapsto O]$.

Solution: To show that $f_P[a := b](\sigma)$ is not locally sound, we need to find a counterexample where the premises of local soundness

$$(1) \quad P \vdash c_i \rightsquigarrow c_{i+1}, \quad (2) \quad \alpha(c_i) \sqsubseteq \sigma_{n_i}, \quad \text{and} \quad (3) \quad f[P[n_i]](\sigma_{n_i}) = \sigma_{n_{i+1}}$$

hold but the conclusion $\alpha(c_{i+1}) \sqsubseteq \sigma_{n_{i+1}}$ does not.

Suppose we have a program P such that

$$P[n_i] = a := 8.$$

Suppose that $c_i = \langle E_i, n_i \rangle$, $c_{i+1} = \langle E_{i+1}, n_{i+1} \rangle$ such that $P \vdash c_i \rightsquigarrow c_{i+1}$. By inversion on the small-step semantics of the assignment statement in WHILE3ADDR,

$$E_{i+1} = E_i[a \mapsto 8] \quad \text{and} \quad n_{i+1} = n_i + 1.$$

Indeed, such P , c_i , and c_{i+1} satisfy the first premise

$$(1) \quad P \vdash c_i \rightsquigarrow c_{i+1}.$$

Suppose a σ_{n_i} satisfies the second premise

$$(2) \quad \alpha(c_i) \sqsubseteq \sigma_{n_i}.$$

Then, for the third premise,

$$(3) \quad \sigma_{n_{i+1}} = f_P[a := b](\sigma_{n_i}) = \sigma_{n_i}[a \mapsto O]$$

by applying the flow function.

Indeed,

$$\begin{aligned}\alpha(c_{i+1}) &= \alpha(E_{i+1}) \\ &= \alpha(E_i[a \mapsto 8]) \\ &= \alpha(E_i)[a \mapsto V] \quad (\text{since 8 is even}) \\ &\neq \sigma_{n_i}[a \mapsto O] \\ &= \sigma_{n_{i+1}}.\end{aligned}$$

□

For the following question, consider the following flow function for parity analysis:

$$f_P[a := b * c](\sigma) = \begin{cases} \sigma[a \mapsto V] & \text{if } \sigma(b) = V \vee \sigma(c) = V \\ \sigma[a \mapsto \perp] & \text{if } \neg(\sigma(b) = V \vee \sigma(c) = V) \wedge (\sigma(b) = \perp \vee \sigma(c) = \perp) . \\ \dots & \text{other cases not specified} \end{cases}$$

2. While the provided case is mathematically correct (an even number multiplied by anything is still even), it is not monotonic. Prove it.

Solution: To prove that the flow function is not monotonic, we need to find two abstract states σ_1 and σ_2 such that $\sigma_1 \sqsubseteq \sigma_2$ but $f_P[a := b * c](\sigma_1) \not\sqsubseteq f_P[a := b * c](\sigma_2)$.

Let $\sigma_1 = \{a \mapsto \top, b \mapsto V, c \mapsto \perp\}$ and $\sigma_2 = \{a \mapsto \top, b \mapsto \top, c \mapsto \perp\}$.

We can verify that $\sigma_1 \sqsubseteq \sigma_2$, but

$$f_P[a := b * c](\sigma_1) = \sigma_1[a \mapsto V] \not\sqsubseteq \sigma_2[a \mapsto \perp] = f_P[a := b * c](\sigma_2),$$

which shows that the flow function is not monotonic. □

When designing dataflow analyses, improper handling of $\perp$ can often lead to non-monotonic flow functions.

For the next two questions, use the following (correct) flow function for parity analysis:

$$f_P[a := b * c](\sigma) = \begin{cases} \sigma[a \mapsto \perp] & \text{if } \sigma(b) = \perp \vee \sigma(c) = \perp \\ \sigma[a \mapsto O] & \text{if } \sigma(b) = O \wedge \sigma(c) = O \\ \sigma[a \mapsto V] & \text{if } (\sigma(b) = V \wedge \sigma(c) \neq \perp) \vee (\sigma(b) \neq \perp \wedge \sigma(c) = V) \\ \sigma[a \mapsto \top] & \text{if } (\sigma(b) = \top \wedge \sigma(c) \notin \{V, \perp\}) \vee (\sigma(b) \notin \{V, \perp\} \wedge \sigma(c) = \top) \end{cases}$$

3. Prove the monotonicity of $f_P[a := b * c](\sigma)$ for the case $(\sigma(b) = V \wedge \sigma(c) \neq \perp) \vee (\sigma(b) \neq \perp \wedge \sigma(c) = V)$

Solution: Here we outline the overall proof which gets split into different cases. Then we go into more details in one case.

Suppose

$$\sigma_1 \sqsubseteq \sigma_2. \quad (1)$$

We want to show that $f(\sigma_1) \sqsubseteq f(\sigma_2)$.

To do so, we can case on the different possible ways of computing $f(\sigma_1)$. If, for each possible ways of computing $f(\sigma_1)$, we have $f(\sigma_1) \sqsubseteq f(\sigma_2)$, then we have proven $f(\sigma_1) \sqsubseteq f(\sigma_2)$ in all cases.

Because $f_P[a := b * c](\sigma)$ has four cases, there are four cases of computing $f(\sigma_1)$. Here, we will go through the third case when

$$(\sigma_1(b) = V \wedge \sigma_1(c) \neq \perp) \vee (\sigma_1(b) \neq \perp \wedge \sigma_1(c) = V).$$

Observe that this is actually two sub-cases, namely

$$\sigma_1(b) = V \wedge \sigma_1(c) \neq \perp$$

and

$$\sigma_1(b) \neq \perp \wedge \sigma_1(c) = V$$

where the latter sub-case is symmetric to the former. We will focus on the first sub-case.

Case $\sigma_1(b) = V$ and $\sigma_1(c) \neq \perp$. By the definition of f_P , we get

$$f(\sigma_1) = \sigma_1[a \mapsto V].$$

Now, what does $\sigma_1(b) = V \wedge \sigma_1(c) \neq \perp$ imply for σ_2 given that $\sigma_1 \sqsubseteq \sigma_2$? Since $\sigma_1(b) \sqsubseteq \sigma_2(b)$ and $\sigma_1(c) \sqsubseteq \sigma_2(c)$ because of (1), it follows that

$$\sigma_2(b) \in \{V, \top\} \quad \text{and} \quad \sigma_2(c) \neq \perp$$

by looking at the lattice.

We now compute $f(\sigma_2)$. Because $\sigma_2(b) \in \{V, \top\}$ and $\sigma_2(c) \neq \perp$, applying the definition of f_P , we get

$$f(\sigma_2) \in \{\sigma_2[a \mapsto V], \sigma_2[a \mapsto \top]\}$$

Clearly, $\sigma_1[a \mapsto V] \sqsubseteq \sigma_2[a \mapsto V]$ and $\sigma_1[a \mapsto V] \sqsubseteq \sigma_2[a \mapsto \top]$ because $\sigma_1 \sqsubseteq \sigma_2$ (because of (1)), $V \sqsubseteq V$, and $V \sqsubseteq \top$, which shows that $f(\sigma_1) \sqsubseteq f(\sigma_2)$ in this sub-case.

Case $\sigma_1(b) \neq \perp$ and $\sigma_1(c) = V$. This case is symmetric to the previous one, and the proof follows similarly.

Other cases of computing $f(\sigma_1)$ proceed similarly. In short, given a case of computing $f(\sigma_1)$, use the assumption (1) to arrive at a set of possible values for $f(\sigma_2)$ and show that $f(\sigma_1) \sqsubseteq f(\sigma_2)$ for each possible value of $f(\sigma_2)$.

Since the cases of computing $f(\sigma_1)$ is exhaustive, we have shown that $f(\sigma_1) \sqsubseteq f(\sigma_2)$ in all cases. $\square$

4. Prove the local soundness of $f_P \llbracket a := b * c \rrbracket(\sigma)$

Solution: Suppose

$$(1) \quad P \vdash c_i \rightsquigarrow c_{i+1}, \quad (2) \quad \alpha(c_i) \sqsubseteq \sigma_{n_i}, \quad \text{and} \quad (3) \quad f \llbracket a := b * c \rrbracket(\sigma_{n_i}) = \sigma_{n_{i+1}}$$

where $c_i = \langle E_i, n_i \rangle$ and $c_{i+1} = \langle E_{i+1}, n_{i+1} \rangle$. We want to show that

$$\alpha(c_{i+1}) \sqsubseteq \sigma_{n_{i+1}}.$$

The general strategy is to analyze the concrete execution states. We show one possible case and leave the rest as exercise. In short, if all cases of the concrete execution satisfy local soundness for f_P , then f_P is locally sound.

Case $E_i(b)$ is even. Then, $E_{i+1} = E_i[a \mapsto m]$ where m is an even number. This is by inversion on assumption (1):

$$\frac{P[n_i] = a := b * c \quad E_i(b) \times E_i(c) = m}{P \vdash \langle E_i, n_i \rangle \rightsquigarrow \langle E_i[a \mapsto m], n_i + 1 \rangle} \text{ step-arith}$$

and because $E_i(b)$ is even, m is also even.

Now, we analyze the abstract execution. Since $E_i(b)$ is even, $\alpha(c_i)(b) = \alpha(E_i(b)) = V$. Hence, because $\alpha(c_i) \sqsubseteq \sigma_{n_i}$, we have

$$\sigma_{n_i}(b) \in \{V, \top\}.$$

Also because $\alpha(c_i) \sqsubseteq \sigma_{n_i}$, and because $\alpha(c_i)(c) \in \{V, O\}$, we have

$$\sigma_{n_i}(c) \in \{V, O, \top\}$$

as well.

Hence, we compute

$$\sigma_{n_{i+1}} = f \llbracket a := b * c \rrbracket(\sigma_{n_i}) \in \{\sigma_{n_i}[a \mapsto V], \sigma_{n_i}[a \mapsto \top]\}$$

as possible values of $\sigma_{n_{i+1}}$.

We also compute

$$\alpha(c_{i+1}) = \alpha(E_{i+1}) = \alpha(E_i[a \mapsto m]) = \alpha(E_i)[a \mapsto V] = \alpha(c_i)[a \mapsto V].$$

Therefore, $\alpha(c_{i+1}) \sqsubseteq \sigma_{n_{i+1}}$ holds because $\alpha(c_i) \sqsubseteq \sigma_{n_i}$ and $V \sqsubseteq V$ and $V \sqsubseteq \top$.

Case $E_i(b)$ is odd and $E_i(c)$ is even. If we flip b and c , the argument of the previous case applies symmetrically. The details are omitted here.

Case $E_i(b)$ is odd and $E_i(c)$ is odd. Then, $E_{i+1} = E_i[a \mapsto m]$ where m is an odd number. The proof proceeds similarly.

□